Requirements Refinery — Architecture Whitepaper

An adversarial-panel system that turns ambiguous requirements into evidence-grounded specifications. Six role-specialised agents debate each requirement under a fused rules + LLM evaluation gate; outputs flow into a five-kind institutional memory store that informs every future debate.

Author	Kevin Quon — linkedin.com/in/kwkwan00
Architecture pattern	Adversarial multi-agent panel with bounded debate, layered-sourcing research, and fused deterministic + probabilistic evaluation
Per-requirement orchestration	LangGraph StateGraph subgraph (generator → critic fan-out → synthesize → score → router → finalize / reconcile / research / escalate)
Panel composition	6 role-specialised seats (Product, Engineering, Security, Operations, Cost, Judge) + 1 search capability (Research) — see Section 3
Evaluation gate	Three-phase combined pipeline: deterministic rules → LLM judge with rule findings in prompt → rule-to-dimension penalty clamp
Knowledge stores	Neo4j (graph + memory labels) + Qdrant (dense + BM25 hybrid retrieval) + Postgres (seven refinery_* forensic tables)
Institutional memory	5 curated kinds — Decision / Incident / Pattern / Constraint / Conflict — with role-filtered recall and atomic write-back on user Apply
LLM provider routing	Single-shot helper picks Anthropic SDK or OpenAI Responses by model id prefix; emits started / ready / failed events per call to the global broker + the SSE stream
Streaming protocol	Server-Sent Events for live per-agent activity (Refinery progress timeline + Agent Log tab share the same event stream)
Operational mode	Assistant — operator reviews and explicitly Applies each requirement and its auto-save-candidate memories
Agentic level	L4 — Fully Autonomous / Explorer (Vellum scale)

Business Value

What the Requirements Refinery delivers — in plain terms

The Requirements Refinery turns ambiguous, underspecified, or naïvely-written requirements into evidence-grounded specifications the downstream pipeline can act on without human re-edit. Operators submit a historical run, a folder of business documents, or a single typed requirement, and receive back refined requirements with explicit acceptance criteria, declared relationships to other requirements, suggested spec decompositions, and a documented decision trail.

Rather than relying on a single agent to do everything, the refinery composes a six-seat **adversarial panel** — a Product seat that drafts, four critic seats (Engineering, Security, Operations, Cost) that challenge from their distinct perspectives, and a Judge seat that synthesises and scores. The panel debates each requirement under an evaluation gate that combines deterministic rules with a separate semantic judge, and a Research capability fetches external evidence on demand, always preferring internal knowledge before reaching for the public web.

CORE FEATURES FOR STAKEHOLDERS

Three input modes	Refine the evidence bundle from a historical pipeline run, ingest a folder of business documents in any common format, or submit a single typed requirement directly. The same panel runs in all three; cross-requirement review automatically skips when only one requirement is in play.
Six-seat adversarial panel	Product drafts; Engineering, Security, Operations, and Cost critique in parallel from their owned perspectives; the Judge writes a revised draft that preserves disagreement in an explicit tradeoffs ledger rather than smoothing it. When two roles propose conflicting fixes, the rationale for picking one over the other is recorded inline so reviewers can audit the decision.

Bounded debate with circuit breaker	Hard caps on debate rounds, model escalations, and external research excursions guarantee that every refinement terminates. When the panel cannot reach agreement within the budget, a dedicated reconcile step documents what stayed unresolved instead of forcing a synthesis that doesn't exist — operators see the open questions explicitly rather than implicitly.
Fused evaluation gate	A deterministic rule layer runs first, catching hard-constraint breaks the language model might wave through. The semantic judge then scores against those same findings, and a final per-dimension penalty caps any score the rules have already invalidated. Three independent signals must agree before a refinement is declared converged.
Layered research capability	When the panel flags missing external information, a four-role research pipeline walks six source tiers in descending trust order: structured internal knowledge first, then internal documents, observability data, official vendor docs, academic sources, and the public web last. Internal knowledge short-circuits external excursions, and any open-web finding requires independent corroboration before it can influence the panel.
Institutional memory write-back	Validated outputs flow back into a structured memory store covering decisions, incidents, patterns, constraints, and conflicts. The memory write commits atomically alongside the user's Apply, so the next debate begins with the system's prior reasoning loaded under each role's filter policy. The system gets measurably faster and cheaper the more it runs.
Per-agent observability	Every model call surfaces in real time with the agent's role, the model used, tokens consumed, latency, and a preview of both the prompt and the response. Three stores capture the data: a forensic SQL layer for joinable analysis, a time-series layer for dashboards and alerts, and a per-debate archive for one-shot replays.
Assistant, not autonomous	The refinery suggests; the user explicitly applies each requirement and any candidate memories proposed by the panel. Requirement identifiers are preserved on edit so existing links between specs and requirements survive every change — applying a refinement never breaks downstream traceability.

> **WHO THIS IS FOR**
>
> Engineering teams that want a structured, defensible answer to "is this requirement ready to spec?" — with the rationale, tradeoffs, and unresolved tensions written down in a form operators can audit and apply selectively.

1. Design Philosophy

The refinery is built on a single premise: **a panel of role-specialised agents arguing under a fused deterministic + probabilistic evaluation gate produces higher-quality refinement than a single agent with a larger context window.** Every architectural choice flows from that premise.

WHY SPECIALISATION BEATS SCALE

A single agent operating on an ambiguous requirement tends toward two failure modes. It *smooths disagreement* — one perspective dominates and tradeoffs go unrecorded — and it *conflates dimensions*, papering over feasibility or risk-coverage gaps with a "looks fine" judgment on clarity. The refinery answers both by giving each panel seat one job and one owned dimension, with its own model, prompt, and slice of context. Engineering challenges feasibility, Security challenges risk coverage, Operations challenges completeness, Cost challenges cost-shaped feasibility, and the Judge — and only the Judge — synthesises.

DISAGREEMENT AS A FIRST-CLASS OUTPUT

Adversariality is enforced structurally, not in the prompt. Critics that produce "looks good" output are rejected by a schema validator and retried with an explicit "you returned a rubber-stamp" follow-up; a second failure becomes a placeholder critique that the trace records as such. The Judge's synthesis is engineered to *preserve* tradeoffs in an explicit ledger rather than smooth them, and a dedicated reconcile step fires on non-convergence to document what the panel could not resolve. When the refinery emits a converged requirement, it means every evaluation signal agreed; when it emits a short-circuited one, operators get a precise ledger of what stayed open.

FULL CONTEXT FOR ADVERSARIES, LIMITED CONTEXT FOR SEARCHERS

A subtle architectural split: the panel seats receive the full shared evidence in one shot. They argue best when every role sees the same picture. The Research capability, by contrast, walks a sequence of tiered sources with intentionally narrow per-tier context — search is a different cognitive task than synthesis, and feeding a searcher every prior page degrades, rather than improves, the next decision. Adversaries get the same map; searchers get one quadrant at a time.

THREE LAYERS OF AGREEMENT BEFORE CONVERGENCE

Convergence is not a single verdict. A deterministic rule layer runs first — fast, free, and immune to model drift. The semantic judge then scores against those same rule findings, instructed to stay consistent with them. Finally a penalty clamp caps any dimension a blocker rule has already invalidated, so the semantic layer cannot overrule a deterministic finding even by accident. Convergence requires an overall score above threshold, every dimension above its own threshold, and zero blockers — three signals that must agree.

> **BOUNDED BY CONSTRUCTION**
>
> Every loop in the debate increments a counter compared to a finite cap, so non-convergence cannot silently loop. Belt-and-braces: the orchestrator carries a hard ceiling on total step count derived from the round budget, so even a pathological branch sequence terminates in a small, predictable number of steps.

2. System Overview

The four-phase SSE pipeline

A refinery invocation is a four-phase Server-Sent-Events pipeline orchestrated by `run_refinery_stream`. Phase 1 assembles the evidence and the requirements list; Phase 2 runs the per-requirement debate (this is where the panel does its work); Phase 3 reviews the refined set cross-requirement; Phase 4 persists results and yields a `done` event with the full `RefineryResponse` payload.

Phase 1 — Gather	Collect requirements from one of three input modes: a historical run (traceability + gaps + episodes + memories + evals + run stats), uploaded documents (parsed via the IngestStage), or a single typed requirement wrapped in a minimal run_context with source_mode='direct'.
Phase 2 — Refine	Each requirement is refined by the LangGraph debate graph in its own worker thread, up to MAX_CONCURRENT_AGENTS (5) in parallel. The graph's per-agent events stream back to the SSE consumer via a thread-local progress callback installed for each worker, so concurrent debates don't bleed events.
Phase 3 — Reconcile	Runs only when the refined set has more than one requirement. The Judge's review_set verb (currently a structural pass-through; LLM-backed cross-set review is a follow-up) feeds apply_cross_review_report which patches relationships, priorities, and duplicate-pair edges back onto the set.
Phase 4 — Persist	The full RefineryResponse is serialised to S3 or local storage at refinery/{result_id}/ alongside metadata.json and a rendered REPORT.md. The SSE stream then yields a single done event with the response payload so the UI can transition from progress to results.

THREE INPUT MODES

Phase 1 accepts `run_id` (refine evidence from a historical pipeline run), `input_path` (refine from uploaded MD/PDF/DOCX/etc), or `direct` (one typed requirement). The direct mode wraps the single requirement in a minimal `run_context` with `source_mode="direct"` and skips Phase 3 because cross-req review is meaningless on a single-item set.

WORKER ISOLATION

Phase 2 dispatches each requirement to its own worker thread via a `ThreadPoolExecutor`. Two pieces of per-thread state keep concurrent debates from bleeding into each other: the SSE orchestrator installs a thread-local progress callback so per-agent LLM events stream back to the right consumer, and each debate's evidence bag carries its own memory accumulator.

3. The Adversarial Panel

Six panel seats and one search capability

Every panel seat subclasses the `RoleAgent` ABC. The base class declares six verbs — `propose`, `critique`, `defend`, `reconcile_unresolved`, `score`, `review_set` — each defaulting to `NotImplementedError`. Each concrete role overrides only the verbs it owns, so the orchestrator can't accidentally call the wrong one (e.g. `EngineeringRole.defend` raises loudly).

Product (generator)	Drafts the first-pass requirement using the full shared context (other requirements + run evidence) baked into one prompt. Single-shot LLM call; no tools, no iteration.
Engineering (critic)	Owns the feasibility dimension. Challenges architecture, scalability, and stack-fit assumptions; pulls from Pattern + Mistake memories under a role-filter policy.
Security (critic)	Owns risk_coverage. Pulls Incident + Constraint memories tagged security/auth/privacy/compliance; identifies abuse cases, vulnerabilities, and compliance gaps.
Operations (critic)	Owns completeness. Pulls Solution + Incident + Pattern memories tagged ops/observability/runbook/incident; evaluates reliability, observability, and failure-mode coverage.
Cost (critic)	Owns cost-shaped feasibility. Pulls Constraint + Pattern memories tagged infra/cost/pricing; explicitly excludes security/privacy memories so cost reasoning isn't biased by unrelated context.
Judge (synthesis + verdict)	Owns all five dimensions. Three verbs: defend (synthesizes the round's critiques into a revised draft, preserving tradeoffs), score (runs the combined evaluation gate), reconcile_unresolved (short-circuit synthesis on non-convergence), review_set (cross-requirement Phase 3).
Research (capability, not a seat)	Invoked by the router only when the Judge flags missing_external_info AND the per-debate research_call_cap has budget remaining. Runs the four-role Librarian → Analyst → Editor → Historian pipeline across six tiered providers — see Section 6.

DEFAULT MODEL TIERS

Each seat picks its default model from one of three tiers, chosen so a false-negative would be most costly where the model is strongest. Within a tier, every seat shares the same default — flipping that default upgrades or downgrades the whole group at once via `refinery_role_models`.

Flagship reasoning tier	Security and Judge. The Security seat owns the highest blast radius (false-negative on a vulnerability beats a noisy false-positive every time); the Judge owns synthesis and the verdict. Both run on the strongest reasoning-tier model the registry ships with.
High-volume critic tier	Engineering and Operations. Both fire on every round across every requirement, so a mid-cost model with strong long-context behaviour is the right balance of latency, cost, and depth.
Numeric / retrieval tier	Product, Cost, and Research. Product synthesizes structured JSON from full context; Cost reasons about quantitative tradeoffs; Research orchestrates retrieval across tiered providers. All three share an OpenAI-family default tuned for structured output and tool-style invocations.

REGISTRY + PER-ROLE OVERRIDES

The `RoleRegistry` reads `refinery_role_models` and `refinery_role_reasoning_effort` dictionaries from `PipelineConfig`, instantiates the factory, and threads overrides through `configure(model=..., reasoning_effort=...)` — so swapping a role's model or escalating its reasoning effort is a config-file edit, not a code change. The same `configure` method is what the router uses to switch to the strong-model tier on escalation.

INFORMATION HIDING BY ABC

The orchestrator may see role *names* (strings) and the ABC's six verbs. It may not import concrete role classes, read prompt text, or know retrieval strategies. This is the Parnas property the ABC exists to protect: every concrete role can be rewritten or swapped without touching the graph.

3.5 Single-shot LLM Helper

One helper, two providers, full per-call observability

Every panel seat's `_call_llm` hook routes through `call_refinery_llm`. The helper picks the SDK from the model id prefix: Anthropic-prefixed ids go to the Anthropic Messages API with extended-thinking budget on `high` / `xhigh` reasoning effort; every other id goes to the OpenAI Responses API with `reasoning.effort` normalised to OpenAI's four tiers. The two adversarial seats that share the flagship reasoning tier go to Anthropic; the high-volume critics share an Anthropic mid-tier; the structured-output seats (Product, Cost, Research) go to OpenAI.

PER-CALL EVENT TRIPLE

Each call emits exactly three events to the observability layer. Tests don't reach this helper — `_call_llm` is monkey-patched on the concrete role before the graph runs — so production network paths can never fire from a test.

refinery_llm_started	Carries agent, model, provider, reasoning_effort, prompt_length, prompt_preview (240 chars). Renders as a dim 'PRODUCT …' / 'SEC …' / 'JUDGE …' row in both the Agent Log tab and the Refinery progress timeline.
refinery_llm_ready	Carries agent, model, provider, latency_ms, tokens_in, tokens_out, response_preview (360 chars). Renders as a bright 'PRODUCT ✓' / 'SEC ✓' / 'JUDGE ✓' row with the response preview inline so the operator sees the actual LLM output.
refinery_llm_failed	On exception, before re-raising. Carries error text, provider, latency_ms. Renders as a red 'AGENT ✕' row with the error preview so operators can grep for misconfigured model IDs and provider auth issues.

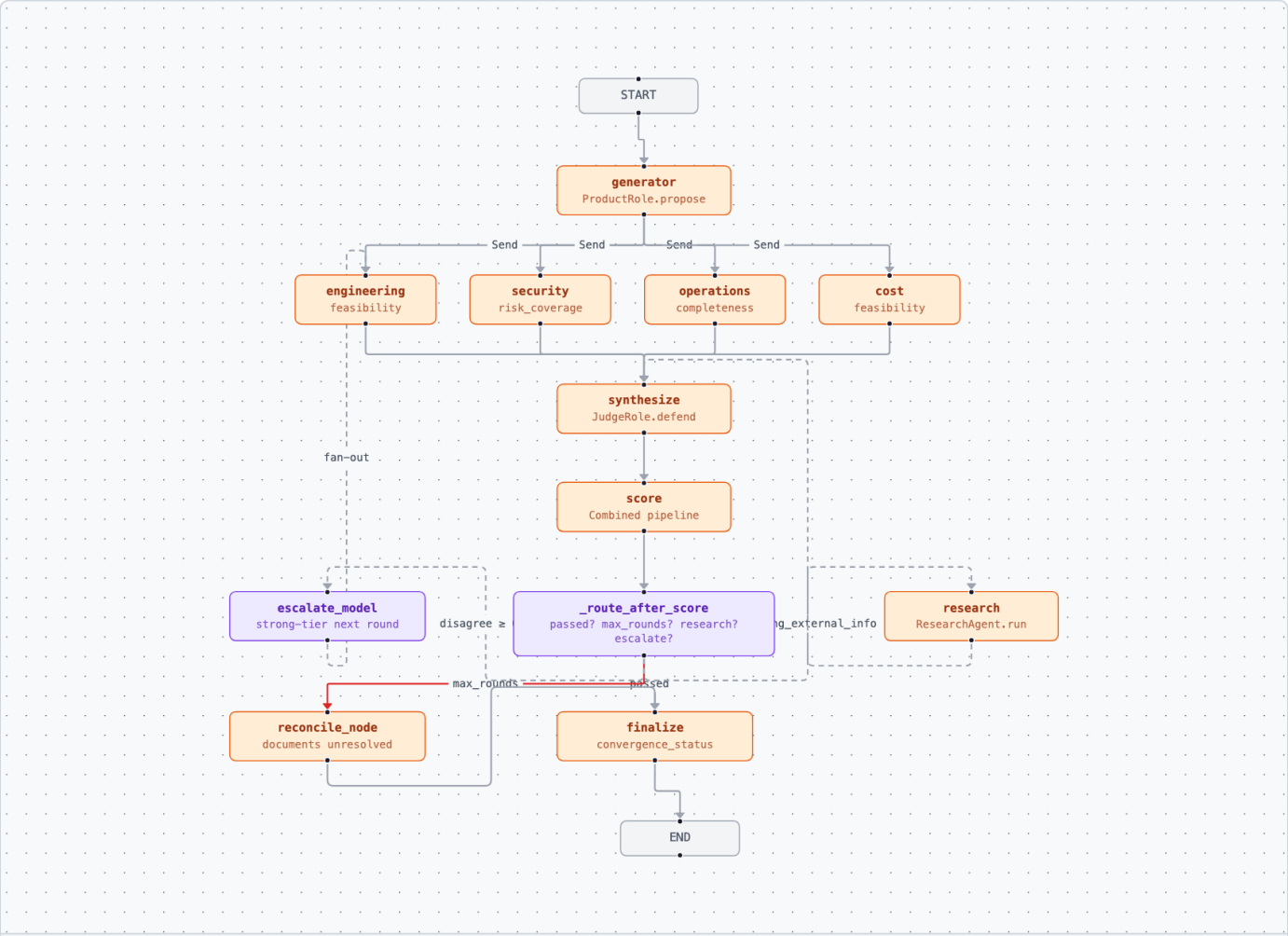
DUAL-ROUTING OF EVENTS

Events are dual-routed by an internal `_emit` helper. The first sink is the global progress broker via `emit_progress` — that's how the Agent Log tab sees swarm and refinery events in one timeline. The second sink is a `threading.local()` slot installed by the SSE orchestrator's `install_progress_callback` context manager — that's how the Refinery progress view sees the live debate inline. Either sink missing degrades gracefully (broker-only or callback-only); neither sink blocks.

4. The Debate Graph

Per-requirement LangGraph StateGraph

`build_debate_graph` wires eight nodes plus the `START` / `END` sentinels. The deterministic spine (generator → critic fan-out → synthesize → score → finalize) runs on static edges; the conditional edges branch on router functions that read the most recent score.



Per-requirement debate graph. The Send fan-out runs the four critics in parallel; the router branches to research / escalate (loops back to synthesize / critics) or to finalize / reconcile (terminal).

STATE SCHEMA

`DebateState` is a `TypedDict` with append-only reducers on every list-shaped field so the parallel critic Sends can write `critiques_by_round` without clobbering each other. The non-obvious entries:

round_number	Incremented only by the synthesizer; bounds the conditional cap. Critics, the score node, and the router all read it but never write it.
research_calls_used / escalation_level	Monotonic counters capped by refinery_research_cap and refinery_escalation_cap respectively. Once exhausted, the router stops branching to those paths.
critiques_by_round	Round → list[Critique]. The barrier reducer merges parallel writes from the four critic Sends so synthesize sees a complete list when its node fires.
research_notes	List[ResearchNote] surfaced into the next synthesize round's evidence bag. The Judge sees ValidatedInsights (with confidence + tier_mix), never raw T5 web snippets.
aborted_reason	Hard abort signal that fast-paths the router to finalize. Set on generator crash or judge error. Ensures even pathological failures terminate cleanly.

THREE TERMINAL OUTCOMES

Converged	passed=True before max_rounds. finalize picks the best-scoring draft; convergence_status='converged'. The user sees the draft + a passing rubric.

Short-circuited	round_number ≥ max_rounds without passing. reconcile_node consolidates what the panel HAS, not what it agreed on, and emits a CONFLICT memory tagged cause='non_convergence'. The frontend renders a 'NOT CONVERGED' badge with the unresolved_points + open_questions surfaced before the description.
Aborted	aborted_reason set (generator crash, judge error). Carry-forward draft; convergence_status='aborted'; no CONFLICT memory because the failure isn't a substantive disagreement, it's a runtime fault.

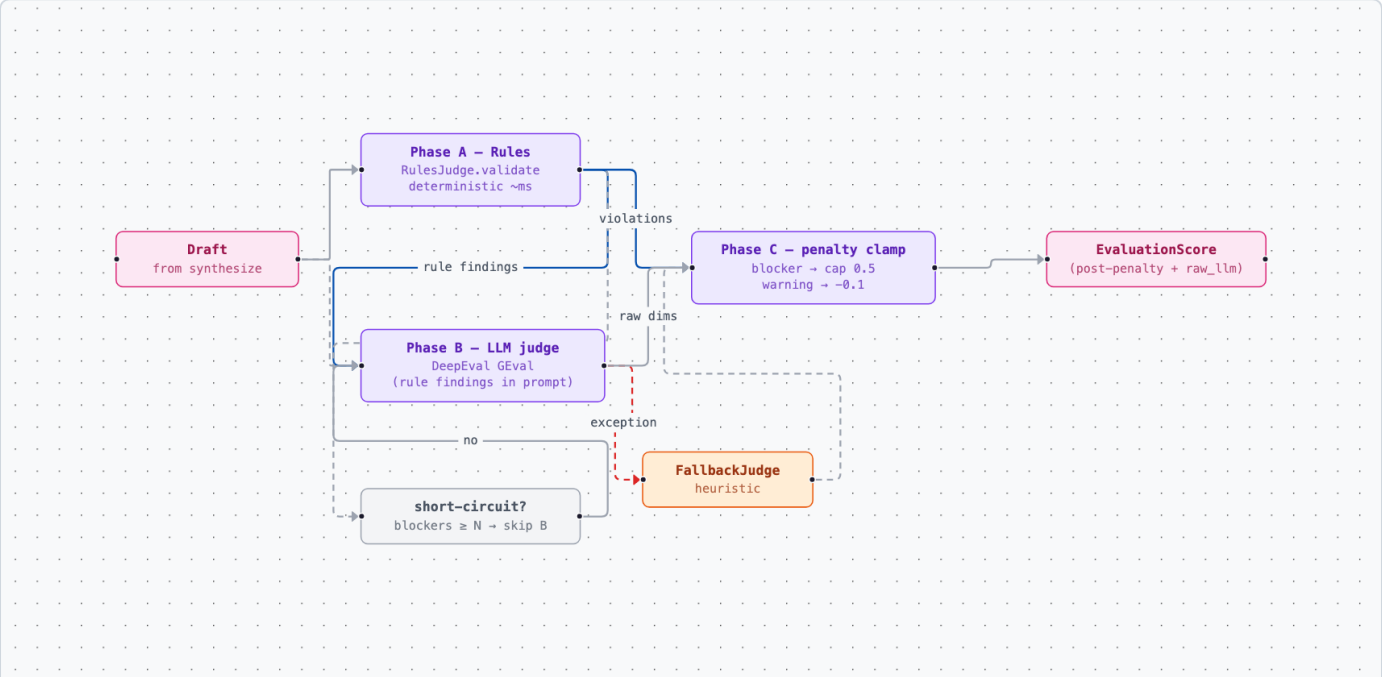
TERMINATION INVARIANTS

Every loop back-edge increments a monotonic counter that is compared to a finite cap in `_route_after_score`. Once `round_number ≥ max_rounds`, the only reachable non-terminal path is `reconcile`, and `reconcile → finalize` is a plain edge with no conditional. Belt-and-braces: `recursion_limit = max_rounds * 8 + 4` on the compiled graph caps node visits at ≈ 27 even when every conditional branch fires.

5. Combined Evaluation Gate

Rules + LLM judge + penalty clamp, one verdict

`CombinedJudgePipeline.score` runs three phases and returns one `EvaluationScore` with both pre-penalty and post-penalty dimension dictionaries so the trace can audit divergence between rule findings and LLM scores.



Combined evaluation pipeline. Phase A's rule findings ride into Phase B's prompt so the LLM scores in line with deterministic findings; Phase C clamps as a final backstop. ``passed`` requires all three to agree.

WHY "FUSED" RATHER THAN "STACKED"

Phase B sees Phase A's findings inside its prompt — the LLM is instructed to score consistently with the rule output rather than judging in isolation. Phase C is the backstop: if the LLM ignores the preamble and returns clarity = 0.95 despite a blocker rule fire, Phase C clamps it to 0.5, the per-dim threshold of 0.7 rejects, and the gate fails. Rules have the final say; the LLM rarely needs the clamp in practice.

RULE CATALOG

Every `RuleSpec` declares its affected dimension as a class-level field, so Phase B can route findings into the right scoring criterion and Phase C can clamp the matching dimension. Starting catalog:

rule_ids_preserved (blocker / completeness)	Draft.requirement_id must equal input.id. Graph-stability invariant — preserves all existing IMPLEMENTS edges from specs.

rule_priority_valid (blocker / feasibility)	Priority must be one of {low, medium, high, critical}.
rule_convergence_consistency (blocker / clarity)	If convergence_status='converged', unresolved_points and open_questions must be empty; if 'short_circuited', at least one must be non-empty.
rule_acceptance_criteria_min_count (blocker / testability)	Each suggested_spec must have ≥ 3 acceptance criteria.
rule_acceptance_criteria_measurable (blocker / testability)	Each criterion must match GIVEN/WHEN/THEN or carry a measurable outcome (number + unit, observable condition). Pure exhortation ('should be fast') fails.
rule_description_specificity (warning / clarity)	Vague modifiers (fast, secure, scalable, robust, simple) without an adjacent concrete measure within ±15 tokens.
rule_relationships_non_circular (blocker / completeness)	Relationship set must not form a cycle when added to the graph.
rule_tradeoffs_required (blocker / feasibility)	If Rebuttal.rejected has ≥ 1 entry citing a conflicting fix, Draft.explicit_tradeoffs must be non-empty.
rule_no_rubber_stamp_rebuttal (blocker / feasibility)	Judge's Rebuttal.rejected entries must cite a value-judgment (rationale ≥ 25 chars). Empty-rationale rejects fail the gate — bridges to the adversarial-contract layer.
rule_t5_confidence_cap (warning / risk_coverage)	Any ValidatedInsight with source_tier_mix=[T5] only must have confidence ≤ 0.30. Belt-and-braces backstop for the research guardrail layer.

> **OPERATOR LEVERS**
> `refinery_rules_disabled` skips named rules (logged to `trace.rules_skipped`); `refinery_rules_short_circuit_llm = True` skips Phase B when ≥ N blocker violations make LLM scoring wasteful (default off because trace value remains useful even when the gate will fail); `refinery_rules_penalty_blocker_cap` and `refinery_rules_penalty_warning_delta` tune the Phase C clamp.

5.5 Adversarial Contract

Disagreement enforced at the data layer, not the prompt

The hardest failure mode of multi-agent debate is **convergent rubber-stamping** — every critic returns generic approval and the system claims unanimous agreement. The refinery rejects this at three layers.

LAYER 1 — ADVERSARIAL PREAMBLE

Each critic's prompt opens with: "your job is to challenge this draft from your role's perspective, not to validate it" plus an explicit escape hatch — return INFO-severity with a non-empty search-narrative if no concern is found after a deliberate search.

LAYER 2 — RUBBER-STAMP VALIDATOR

`is_rubber_stamp` rejects schema-conforming but vacuous output: `finding` shorter than 20 chars, `proposed_fix` shorter than 10 chars, or any of a known-rubber-stamp pattern set ("looks good", "no issues", etc.) — unless severity is INFO.

LAYER 3 — SINGLE RETRY, THEN PLACEHOLDER

`call_critic_with_retry` retries once with a pointed "you returned a rubber-stamp" follow-up. A second failure becomes a placeholder Critique with `error` set, the round continues, and the trace records `placeholder_rubber_stamp = true` so operators can grep for the failure mode.

SYNTHESIS PRESERVES DISAGREEMENT

The Judge's `defend` prompt requires every `Rebuttal.rejected` entry to cite the specific critique AND the value-judgment behind the rejection (e.g. "rejecting Cost's WARNING because Security's BLOCKER outweighs a 2× spend delta on a priority=critical auth feature"). When two critics raise mutually exclusive fixes, the revised `Draft.explicit_tradeoffs` must name the choice — and `rule_tradeoffs_required` is a Phase A blocker that fails the gate when it doesn't.

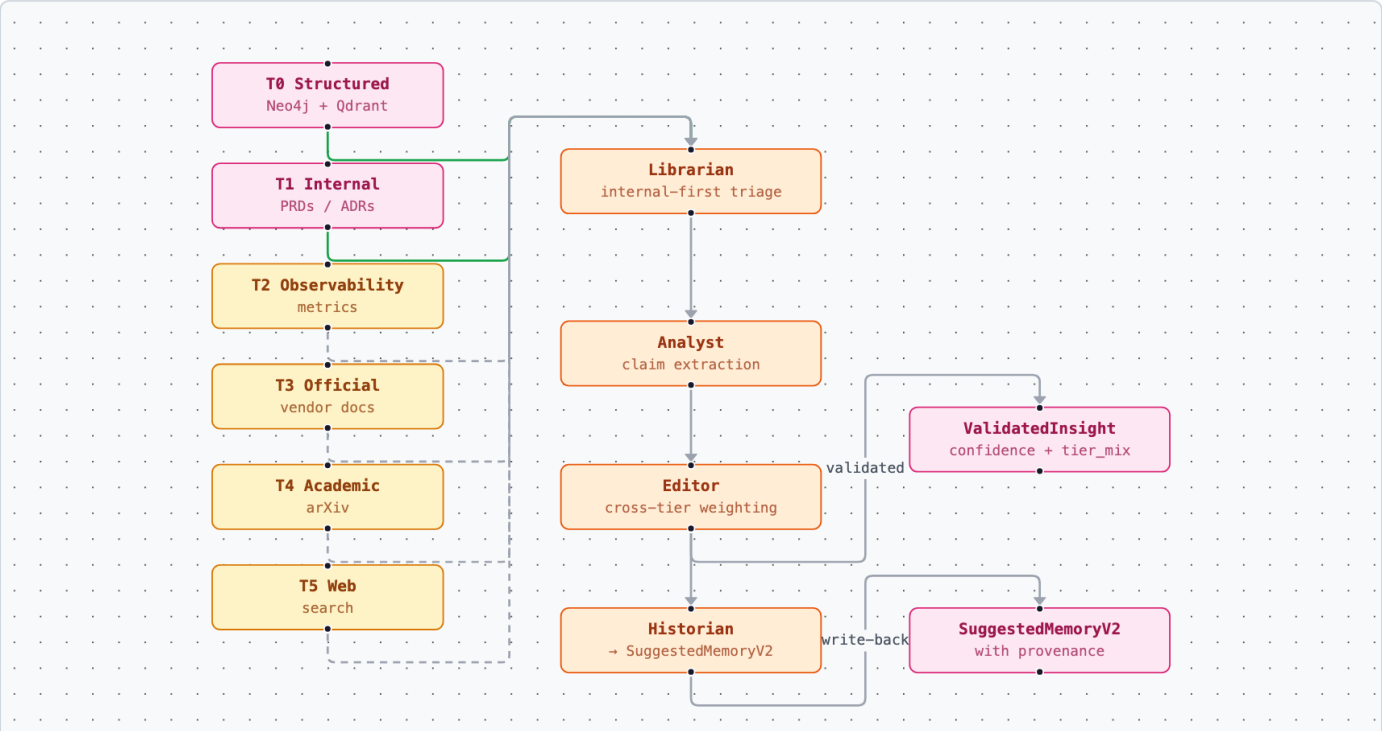
ANTI-COLLUSION ISN'T FREE

A sufficiently-clever prompt could craft "substantive" critiques that all four critics align on, leaving real issues uncovered. Mitigations: per-role distinct dimensions reduce the surface, an anti-collusion test fixture plants flaws and asserts each role finds a different one, and the rules layer catches deterministic gaps even when the critics miss them. The system trusts the rules; it instruments the LLM.

6. Layered Research Agent

Internal-first triage across six tiers, three guardrails

Research is a *capability*, not a panel seat. The router invokes it only when the Judge flags `missing_external_info = True` AND the per-debate `research_call_cap` has budget remaining. Internally, `ResearchAgent.run` walks four roles across six tiered providers.



Layered research pipeline. Solid edges from T0 / T1 are the internal-first short-circuit path; dashed external tiers fire only when the internal signal is below threshold and the per-tier budget has slots remaining.

THE FOUR ROLES

Librarian	Internal-first triage. Hits T0 (Neo4j + Qdrant + trace) and T1 (PRDs / ADRs / postmortems / wikis) first. If best_trust_score ≥ refinery_research_internal_sufficient_threshold (default 0.75), short-circuits — no external tier fires. Otherwise, T2 (observability), T3 (official vendor docs), T4 (academic), T5 (web) run in descending trust order, each capped by its tier budget.
Analyst	ONE LLM call over the fetched corpus. Extracts atomic claims into ExtractedClaim records, each citing source.id (Pydantic-validated). Deliberately tool-less — no follow-up URLs, no per-claim browsing.
Editor	Clusters claims by embedding similarity (≥ 0.85), trust-weights across tiers, and emits ValidatedInsight records. Cross-tier contradictions are preserved in ValidatedInsight.contradicting_claims so the Judge can surface tension instead of picking a side silently.
Historian	Converts ValidatedInsight to SuggestedMemoryV2 with provenance_source_tier_mix, provenance_confidence, provenance_citations. These flow through the institutional-memory write-back loop on user Apply.

THREE ENFORCED GUARDRAILS

T5 raw never propagates	A Pydantic validator on ResearchNote rejects any payload with a T5 citation that isn't wrapped in a ValidatedInsight backed by ≥ 1 non-T5 corroborating source. Web findings cannot reach the Judge unless an internal tier corroborates them.
No free browsing	Each provider exposes search() + fetch(url) only; fetch URLs must match the provider's allowlist. T3 uses a hardcoded vendor list (AWS / Azure / GCP / Anthropic / OpenAI / FastAPI / Qdrant / Neo4j); T5 limits fetch URLs to those returned by its own search.
Per-tier budget exhaustion	Each tier has a slot count from refinery_research_tier_budgets. Further calls raise TierBudgetExhausted, forcing the agent to reason with what it has. Default budgets: T0=8, T1=6, T2=4, T3=4, T4=2, T5=6.

WHY INTERNAL-FIRST MATTERS

When validated insights flow through the write-back loop they become T0 / T1 entries on subsequent runs. The next debate that hits the same query short-circuits at the Librarian — the system *doesn't search; it already knows*. Each external excursion measurably reduces the next one.

6.5 Role-Filtered Retrieval

Information hiding enforced at the data layer

Every retrieval is filtered server-side by the role's `RoleFilterPolicy`, not post-hoc in Python. The policy translator emits a `qdrant_client.models.Filter` with `must` / `must_not` / `should` clauses; a Cypher counterpart constrains the Neo4j graph traversal. A role cannot receive rows outside its slice even if its prompt malfunctions and tries to ask for them.

Product	Memory kinds: decision, pattern, conflict. Graph: RELATED_TO + DEPENDS_ON. Row cap: 12. Receives prior tradeoffs at propose-time so the generator preempts known disagreements.
Engineering	Memory kinds: pattern, incident, constraint, conflict. Excludes security-only tags. Graph: DEPENDS_ON + IMPLEMENTS. Row cap: 15.
Security	Memory kinds: incident, constraint, conflict. Includes security/auth/privacy/compliance/incident tags. Graph: DEPENDS_ON + RELATED_TO. Row cap: 15.
Operations	Memory kinds: incident, constraint, pattern. Includes ops/observability/runbook/incident tags. Graph: IMPLEMENTS + RELATED_TO. Row cap: 12.
Cost	Memory kinds: constraint, pattern. Includes infra/cost/pricing tags; excludes security/privacy. Graph: DEPENDS_ON. Row cap: 10.
Research	empty=True (no Qdrant or Neo4j access). Fetches externally through its own provider stack; nothing the role does is constrained by the local store.
Judge	Reads the full DebateTrace for per-req scoring + the full refined set + all traces for cross-req review_set. No store retrieval; trace-only access by design.

FROZEN CONTEXT + AUDIT FINGERPRINT

A built `RoleContext` is `model_config = ConfigDict(frozen=True)` so a role cannot mutate it to smuggle in extra rows. Each retrieved row is wrapped in a `SourceRef` (collection, point_id, score, matched_filter_fields) before it enters `RoleContext.evidence`; the policy + query are hashed into `context_fingerprint` so traces can recover which filter produced which context for every role call.

OPERATOR OVERRIDES

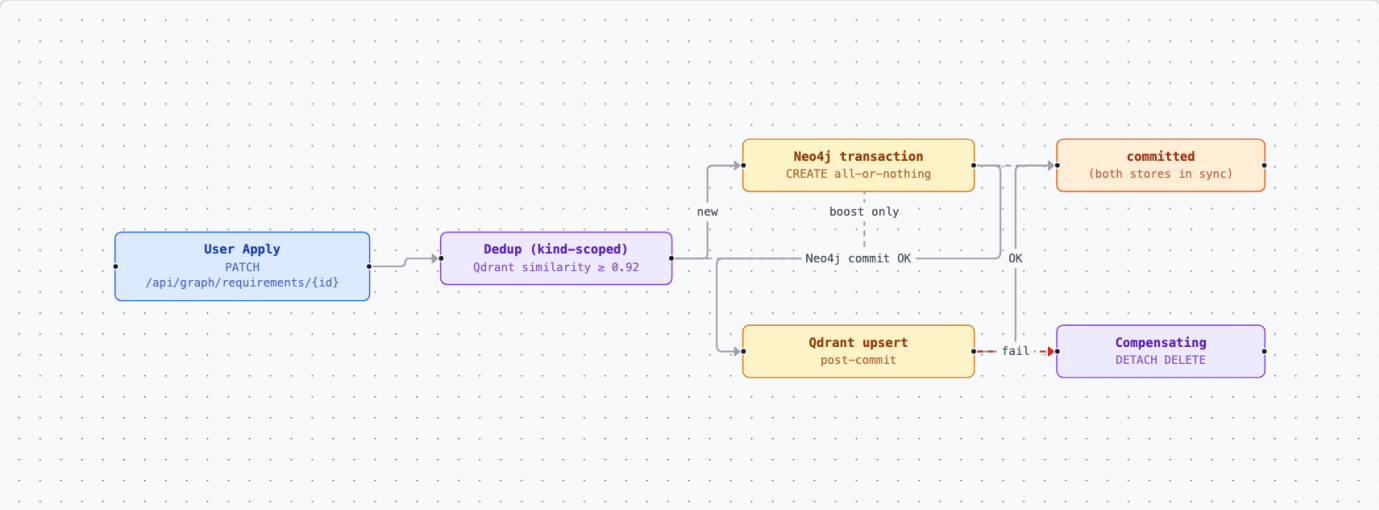
`refinery_role_filter_overrides` is a `dict[str, dict]` of partial `RoleFilterPolicy` overrides applied at registry construction time. Example: bump Security's row_cap to 20 and add "threat-model" to its include-tags list without editing code or restarting the rest of the panel.

7. Institutional Memory

Five curated kinds, kind-scoped dedup, atomic write-back

Memory is the substrate the refinery both produces and consumes. Five kinds are emitted at specific debate events by per-event producer functions:

Decision	Synthesizer accepts/rejects a critique with a generalisable rationale; Judge assigns a verdict at threshold boundary. Consumed by Product (propose) and Judge (defend / score / review_set).
Incident	Critic emits severity=blocker citing a runtime/incident pattern (:Mistake label in Neo4j). Consumed by Security, Operations, Engineering.
Pattern	Generated draft structure matches a reusable template; cross-req Judge sees the same structure across ≥ 2 reqs. Consumed by Product, Engineering.
Constraint	Critic blocker cites a system or business limitation (stack pin, API rate limit, compliance, budget). Consumed by Product, Judge, Cost, Engineering.
Conflict	disagreement_score ≥ refinery_conflict_emit_threshold at finalize OR reconcile_node fires unconditionally on short-circuit (cause='non_convergence'). Consumed by Product (propose), Judge (review_set), all critics (heads-up).



Atomic memory write-back on Apply. Dedup runs read-only outside the transaction. New nodes commit in one Neo4j tx; Qdrant upserts happen post-commit. On Qdrant failure, a compensating DETACH DELETE keeps the two stores in sync.

ATOMIC WRITE-BACK CONTRACT

On user Apply, the request to `PATCH /api/graph/requirements/{req_id}` carries an optional `apply_memories` list. The handler routes through `MemoryRepository.record_refinery_memories_atomic`. Dedup runs read-only outside the transaction (Qdrant similarity ≥ `memory_dedup_threshold`, kind-scoped — a Pattern and a Decision with identical text are NOT considered duplicates of each other). New nodes commit in one Neo4j transaction; Qdrant upserts happen post-commit. On Qdrant failure, a compensating `DETACH DELETE` keeps the two stores in sync.

VALIDATION STATUS GOVERNS AUTO-SAVE

Each suggested memory carries a `validation_status` field. `validated` (Judge cited it in synthesis) and `produced` (generated during a converged debate) auto-save by default; `unvalidated` and `user_dismissed` stay suggested for explicit review. Operators flip `refinery_auto_save_on_apply = False` to restore strict assistant-mode (every memory reviewed individually).

8. Observability

Three stores, one trace, dual-write to legacy llm_calls

Each LLM call, tool call, and routing decision becomes a typed record in `DebateTrace`. The `ObservabilityHub` fans events to three sinks with intentionally distinct retention and query shapes.

Postgres — forensic + joinable	Ten refinery_* tables FK'd back to pipeline_runs: the seven core forensic tables (refinery_runs, refinery_debates with priority+tags columns for archetype bucketing, refinery_debate_rounds, refinery_llm_calls, refinery_rule_violations, refinery_research_sources, refinery_memory_audits), the resume cache (refinery_debate_completions), plus the three operator-decision tables (refinery_memory_feedback, refinery_critique_dispositions, refinery_requirement_decisions). The seven section-9 loops join these forensic + decision tables in different combinations — adaptive max_rounds reads refinery_debates, self-tuning rule severity joins rule_violations to requirement_decisions, critic-dimension credibility groups dispositions by (role, dimension), etc. For SQL questions like 'which rules fire most on Security-tagged requirements this month' or 'is the Judge actually right about what operators apply'.
Prometheus — time-series aggregates	≈ 25 dark_factory_refinery_* series covering convergence outcomes, rounds histogram, judge scores (pre/post-penalty), rule violations by rule_id/severity/dimension, research tier mix, internal-first hit ratio, memory write-back failures, cost + tokens per role, dual-write failures. A pre-built Grafana dashboard at deploy/grafana-dashboards/refinery.json ships 10 starter panels.
Trace JSON — full-fidelity archive	Per-debate file under refinery/{result_id}/trace.json. Captures every prompt, response preview, tool call, routing decision, rule violation. Persistent for one-shot 'show me what happened in this debate' replays.

DUAL-WRITE TO LLM_CALLS

Every refinery LLM call writes to *both* `llm_calls` (legacy system-wide cost ledger) and `refinery_llm_calls` (refinery-specific role / round / req_id detail) inside one transaction. Existing FinOps queries against `llm_calls` automatically include refinery cost; the dashboards don't need a regex tweak to see it. The `phase` column is set to `refinery.<role>.<kind>` (e.g. `refinery.security.critique`) so a single `GROUP BY` reveals per-role spend.

> **PER-AGENT LIVE VIEW**
>
> The LLM helper's three-event triple (started / ready / failed) flows to both the global progress broker and the refinery SSE callback, so the Agent Log tab and the Refinery progress timeline both show the same per-agent activity in real time. Filter by role name ("security", "judge") or event ("refinery_llm_ready") to audit a single agent's activity across an entire run.

9. Continuous Calibration

Seven learning loops + an active-learning feedback path turn operator and panel signal into bounded multipliers on the next run

Seven cooperating subsystems — each owning a distinct signal — nudge the next debate's behaviour based on what happened in prior runs. All seven follow the same architectural pattern: Postgres telemetry table → bounded aggregator → identity default on missing data → multiplier injected into the evidence bag (or runner kwarg, for orchestration parameters) at run start. None can hard-fail the pipeline; a Postgres outage means every loop falls back to `×1.0` (configured base for max_rounds; configured severity for rules) and the system behaves exactly as it did before the loop existed. This is what graduates the refinery from a stateful debate to genuinely L4 — the system observes its own predictions vs. operator follow-through and adjusts.

Provider trust learning	Per-(tier, provider) propagation rate from refinery_research_sources feeds the Editor's confidence weighting. A T5 web provider whose claims keep getting rejected drifts down within its tier envelope; a T3 vendor doc that consistently survives cross-referencing drifts up. Bounds are per-tier so a runaway loop can't turn T5 into T0 or vice versa.

Adaptive role weighting	The synthesize node records every (role, severity, action) triple from the Judge's rebuttal ledger into refinery_critique_dispositions. The aggregator computes per-role acceptance rates over the trailing 30-day window; the resulting multiplier (bounded [0.60, 1.40]) is rendered into format_defend_prompt as a calibration block so the LLM weighs each critic by historical accuracy. Below 8 disposition rows in the window, every role gets identity — early signal isn't trustworthy.
Critic-dimension credibility weighting	Refines the flat per-role multiplier by splitting on the dimension a critique actually cited. The same dispositions table is grouped by (role, dimension); per-cell bounds are [0.50, 1.50] (slightly wider on the high side because a role's findings on its native dimension should outweigh the flat band). Per-role and per-dim multipliers compose multiplicatively, with a hard product clamp [0.50, 1.60] so two factors can't compose into ranges neither was tuned for. The synthesis prompt receives a 2D table of pre-combined values, rendered once per run because role × dim is invariant across rounds of the same debate.
Judge confidence calibration	Operator apply / dismiss / edit decisions on refined requirements land in refinery_requirement_decisions, joined against the Judge's predicted overall score (or convergence_status as a fallback when scores haven't been plumbed end-to-end). The aggregator computes a four-quadrant miscalibration ratio (high-applied, high-dismissed, low-applied, low-dismissed) and emits a bounded [0.85, 1.15] multiplier that scales the effective threshold inside CombinedJudgePipeline.score. Tighter bounds than role-weighting because moving the threshold has cascading effects on rounds-per-debate and cost.
Adaptive max_rounds	Per-archetype (priority, source_mode, primary_tag) convergence history from refinery_debates determines whether each requirement should run with a tighter or looser per-debate cap. Archetypes that consistently converge fast (avg rounds < base × 0.55) get the cap reduced by 1; archetypes whose debates short-circuit ≥30% of the time get +1, ≥55% get +2. Hard floor 2, hard ceiling 6. Plumbed as a runner kwarg into DebateConfig — orchestration parameters (recursion-limit, fanout cost) stay off the evidence bag.
Self-tuning rule severity	Per-rule operator override rate (operator applied a requirement despite a rule blocker) on refinery_rule_violations joined to refinery_requirement_decisions. When ≥50% of operators override a particular rule's blockers (with at least 8 decided firings in the window), the rule is auto-demoted from blocker to warning at runtime — its findings still surface in the trace and apply Phase C's warning-delta penalty, but no longer hard-cap dimensions or gate convergence. Demote-only; warnings are never auto-promoted because they don't generate the override signal.
Active-learning feedback	When an operator dismisses or accepts a suggested memory in the UI, three best-effort sinks fire: telemetry to refinery_memory_feedback (joins the calibration tables on cross-run analysis), Neo4j boost_relevance / demote_relevance on the memory node (re-ranks future role retrieval), and on a reasoned dismissal a Conflict memory tagged cause="user_override" so the next debate sees the prior judgement as institutional context. Per-sink success flags come back so the UI can surface partial success without blocking.

WHY THE SAME PATTERN, SEVEN TIMES

Every loop is a Postgres aggregation followed by a bounded multiplier (or bounded integer, for max_rounds) and an identity default. Repetition is the point — once the pattern is in the codebase, adding an eighth loop (e.g. systemic- issue detection across runs, or per-run goal-conditioned routing) is a copy-and-adapt of an existing module rather than a fresh design exercise. Each loop's bounds are tuned independently because the cost of being wrong differs: a noisy provider can't break a debate (per-tier envelope), a misjudged role still gets heard (multiplier never reaches zero), a miscalibrated Judge threshold cascades into rounds and cost so its bounds are tightest, and a runaway max_rounds bumper is hard-capped at 6 because every extra round multiplies critic-fanout cost.

THE FINGERPRINT THAT TIES THE LOOPS TOGETHER

Five of the seven loops stamp into the same `evidence_bag` the Judge reads off `RoleContext.evidence` (`role_weights` , `role_dim_weights` , `calibration_block` , `judge_threshold_multiplier` , `rule_severity_overrides`). Provider trust feeds the Editor inside the research pipeline. Adaptive max_rounds is plumbed as a runner kwarg because it controls graph topology (recursion-limit, fanout) rather than role synthesis. The orchestrator computes every loop's snapshot once at run start (parallel debates share one snapshot so behaviour is stable across the run), promotes the in-bag signals from `run_context` into the evidence bag inside `run_debate` , and the composition pipeline + prompt formatter pick them up by name.

10. L4 Agentic Classification

Where this sits on the Vellum scale

The Requirements Refinery operates at **L4 — Fully Autonomous / Explorer** on the Vellum agentic behavior scale, by the same definition the rest of the Dark Factory platform uses. All five L4 traits are implemented; the deliberately-bounded scope at the Apply boundary makes the system *operate* as an assistant without changing its agentic architecture.

Persistent state across sessions	The five-kind institutional memory store (Decision / Incident / Pattern / Constraint / Conflict) survives across debates and runs. Per-debate ContextCache is scoped to one invocation and cleared between requirements; trace JSON is archived per debate; the seven refinery_* Postgres forensic tables persist forever and dual-write to llm_calls so existing FinOps queries auto-include refinery cost.
Refines execution from feedback	Two surfaces. Inside one debate: the combined evaluation gate fuses three signals — deterministic rules → LLM judge with rule findings injected into the prompt → Phase C dimension clamp — and the router branches to next_round / research / escalate / reconcile / finalize accordingly. Across runs: the four calibration loops in section 9 turn operator and panel signal into bounded multipliers on the next debate (provider trust, role weighting, Judge threshold, plus active-learning feedback that re-ranks memory retrieval). The system measures its own predictions against operator follow-through and adjusts.
Parallel execution	Within a debate round, the four adversarial critics run in parallel via a LangGraph Send fan-out and merge through a barrier reducer on critiques_by_round. Across the requirement set, MAX_CONCURRENT_AGENTS workers run debates concurrently via a ThreadPoolExecutor; a thread-local progress callback installed per worker keeps per-agent SSE events from bleeding between concurrent debates.
Real-time adaptation	Mid-debate escalation upgrades the panel to a stronger reasoning tier when disagreement_score crosses a configurable threshold. Mid-debate research excursions invoke the layered-sourcing agent when the Judge flags missing_external_info AND the per-debate budget has slots. Mid-debate short-circuit to reconcile_node fires when rounds exhaust without convergence — the panel pivots from synthesis to documenting the unresolved tensions instead of forcing a consensus.
Cross-requirement reconciliation	Phase 3 invokes JudgeRole.review_set across the full refined set + every per-requirement DebateTrace, emitting a CrossReviewReport. The structural patcher applies duplicate-pair edges, coherence-issue notes, missing or invalid relationships, priority inversions, and spec overlaps back onto the requirement set in one pass.

ASSISTANT-MODE DISCIPLINE ≠ LOWER AUTONOMY

The Refinery is operated as an assistant — the user reviews and explicitly Applies each refined requirement, and the suggested-memories checklist gives them per-memory veto before write-back commits. That gate is a *scope choice* at the boundary between the panel and the persistent graph; the panel itself debates, escalates, researches, reconciles, and writes its own trace autonomously inside the loop. The Apply boundary is the difference between L4-as-assistant and L4-as-autonomous-loop; same architecture, different operational mode.

11. Tradeoffs + Open Questions

Where the design pays a price, and what's left to build

COST VS DEPTH

Six panel seats × up to three rounds × ~2k token prompts is substantially more expensive than a single-agent refinery. The Librarian's internal-first triage and the rules short-circuit amortise this — most debates terminate in round 1 with two Phase A blockers caught before any LLM scoring fires. For cost-constrained operators, the levers are `refinery_debate_max_rounds = 1` + `refinery_rules_short_circuit_llm = True` (catches hard-constraint breaks deterministically, skips the LLM gate when the verdict is already obvious).

SPEED VS ADVERSARIALITY

Critic fan-out runs in parallel via LangGraph `Send` primitives — a round's wall-clock latency is bounded by the slowest critic, not the sum. The Judge synthesis + score serialise on top, so a typical converged debate runs four parallel critic calls plus two sequential Judge calls per round (~6× single-agent latency for ~4× the evaluation depth). Acceptable for an assistant-mode refinery where humans Apply; less so for an autonomous closed-loop pipeline.

ADVERSARIAL COLLUSION AT THE PROMPT LEVEL

The rubber-stamp validator catches schema-conforming approval but a clever prompt could craft "substantive" critiques that all four agree on, leaving real issues uncovered. Mitigations: per-role distinct dimensions reduce the surface; an anti-collusion test fixture plants flaws and asserts each role finds a different one; and the rules layer catches deterministic gaps even when the critics miss them.

OPEN: CROSS-SET REVIEW COMPUTE ENVELOPE

Phase 3 runs `JudgeRole.review_set` as a bounded critique-and-ratify loop (default 1 pass; raise `refinery_set_review_max_rounds` for stricter adjudication). The Judge sees the full refined set + every per-requirement `DebateTrace` in one prompt, which is fine for tens of requirements but scales poorly past a few hundred. The clean follow-up is a hierarchical pass — cluster the set first, run review_set per cluster, then a meta-review across cluster reports — gated on the same combined-evaluation primitives so the dimension-scoring contract stays consistent at every scale.

OPEN: RESEARCH PROVIDER BREADTH

The current T3 allowlist hardcodes the major cloud + AI SDK vendor docs. Adding a new T3 source is a config edit plus a provider implementation. T4 is arXiv-only; T5 uses OpenAI Responses' built-in web_search tool. Tavily / Serper / Brave alternates are deliberately deferred until the existing T5 provider hits a documented limit.